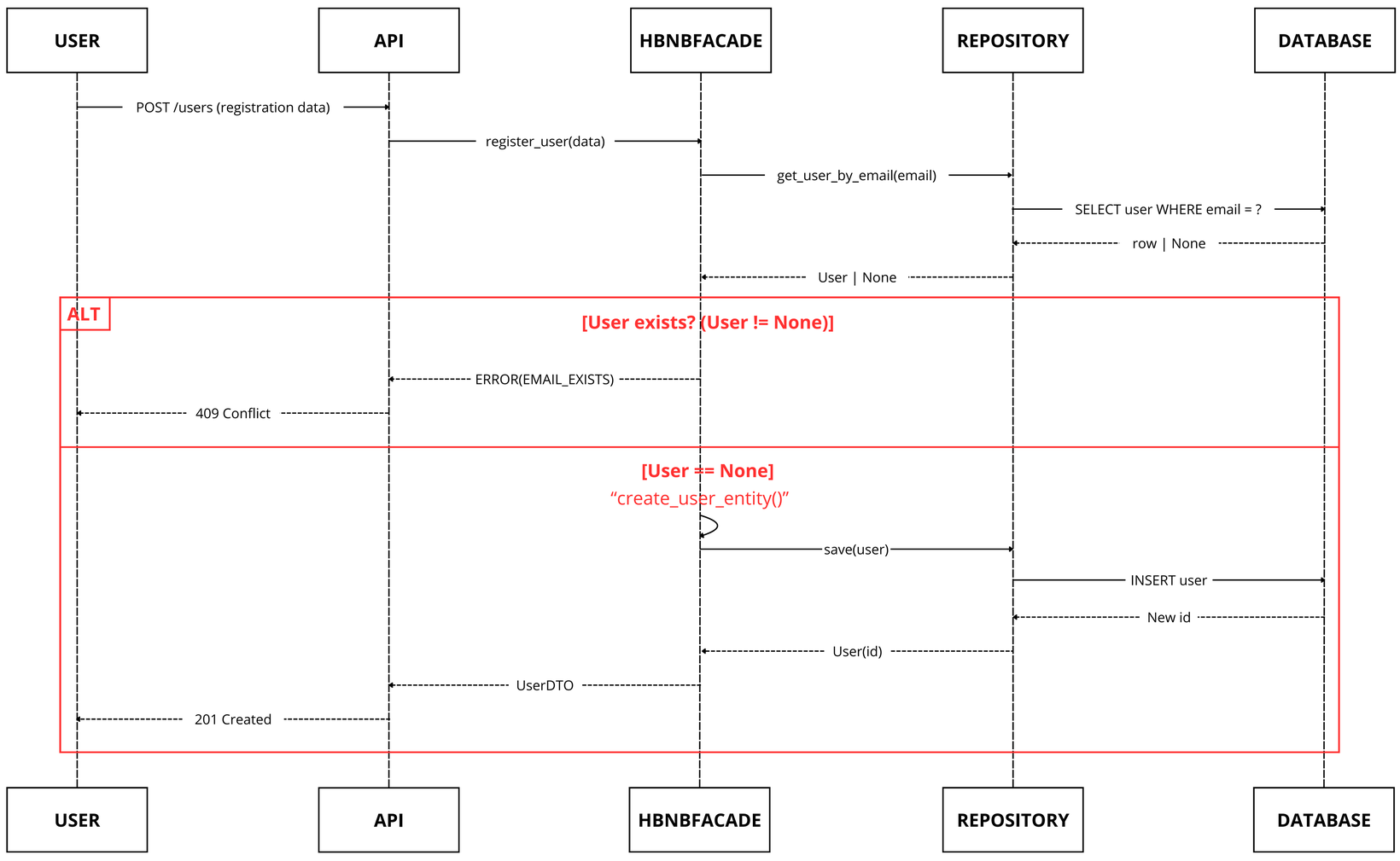
SD-01 — User Registration (POST /users)



Explanatory Notes:

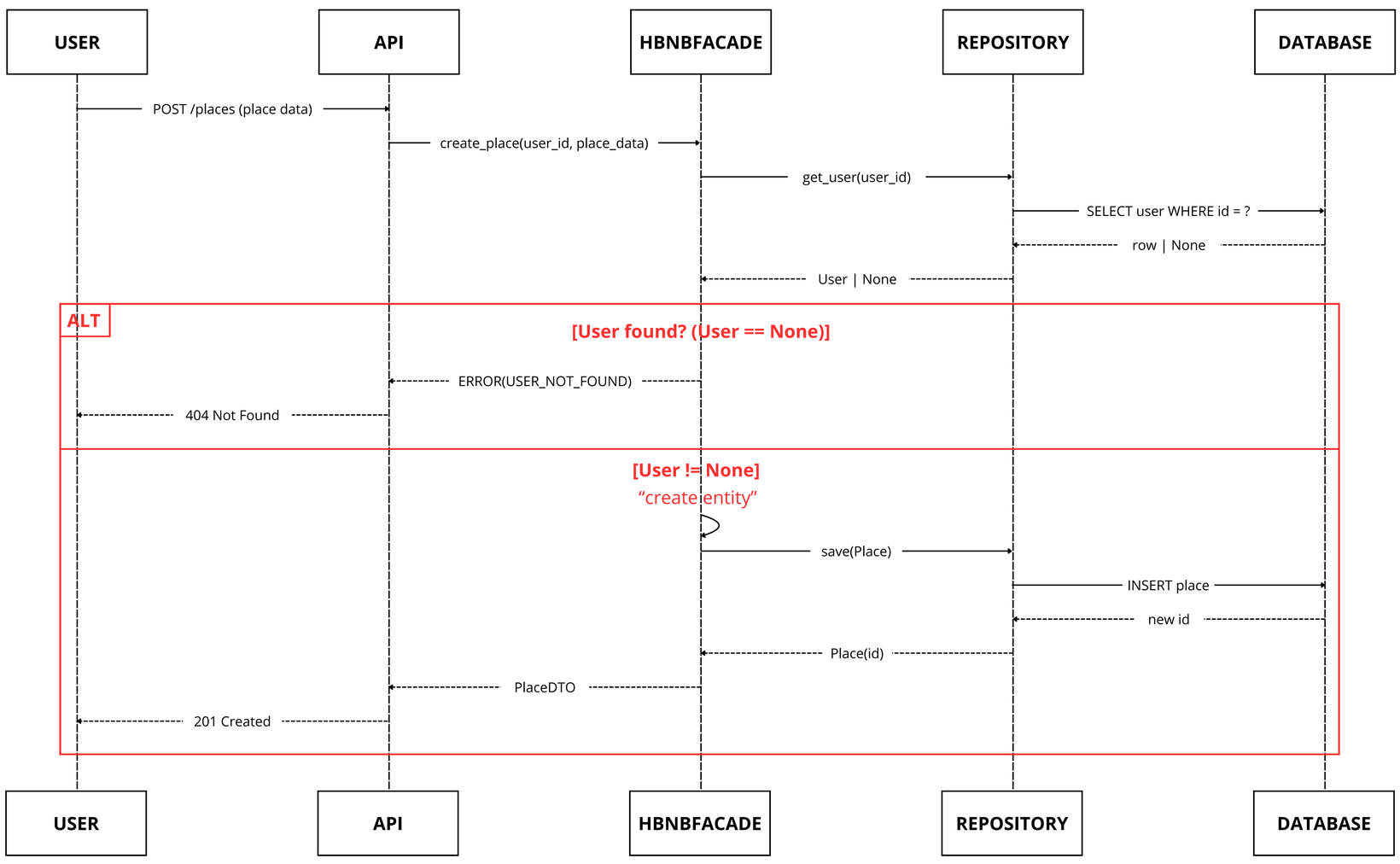
Purpose :
- Register a new user account by validating uniqueness (email) and persisting the user.

Key steps :
- User sends POST /users with registration payload.
- API forwards request to HBnBFacade.register_user(data).
- Facade asks Repository to check if email already exists (SELECT user WHERE email=?).
- If user exists: Facade returns Error(EMAIL_EXISTS) → API returns 409 Conflict.
- If user does not exist: Facade creates User entity, Repository persists it (INSERT user) and returns new id.
- Facade maps persisted entity to UserDTO → API returns 201 Created + UserDTO.

Layer responsibilities :
- Presentation (API): receives request, delegates to facade, returns HTTP response.
- Business (HBnBFacade): orchestrates use case + validation (email uniqueness) + DTO mapping.
- Persistence (Repository/DB): executes SELECT/INSERT, returns rows/new id.

Outputs / errors :
- Success: 201 Created + UserDTO
- Failure: 409 Conflict (email already exists)

SD-02 — Place Creation (POST /places)



Explanatory Notes:

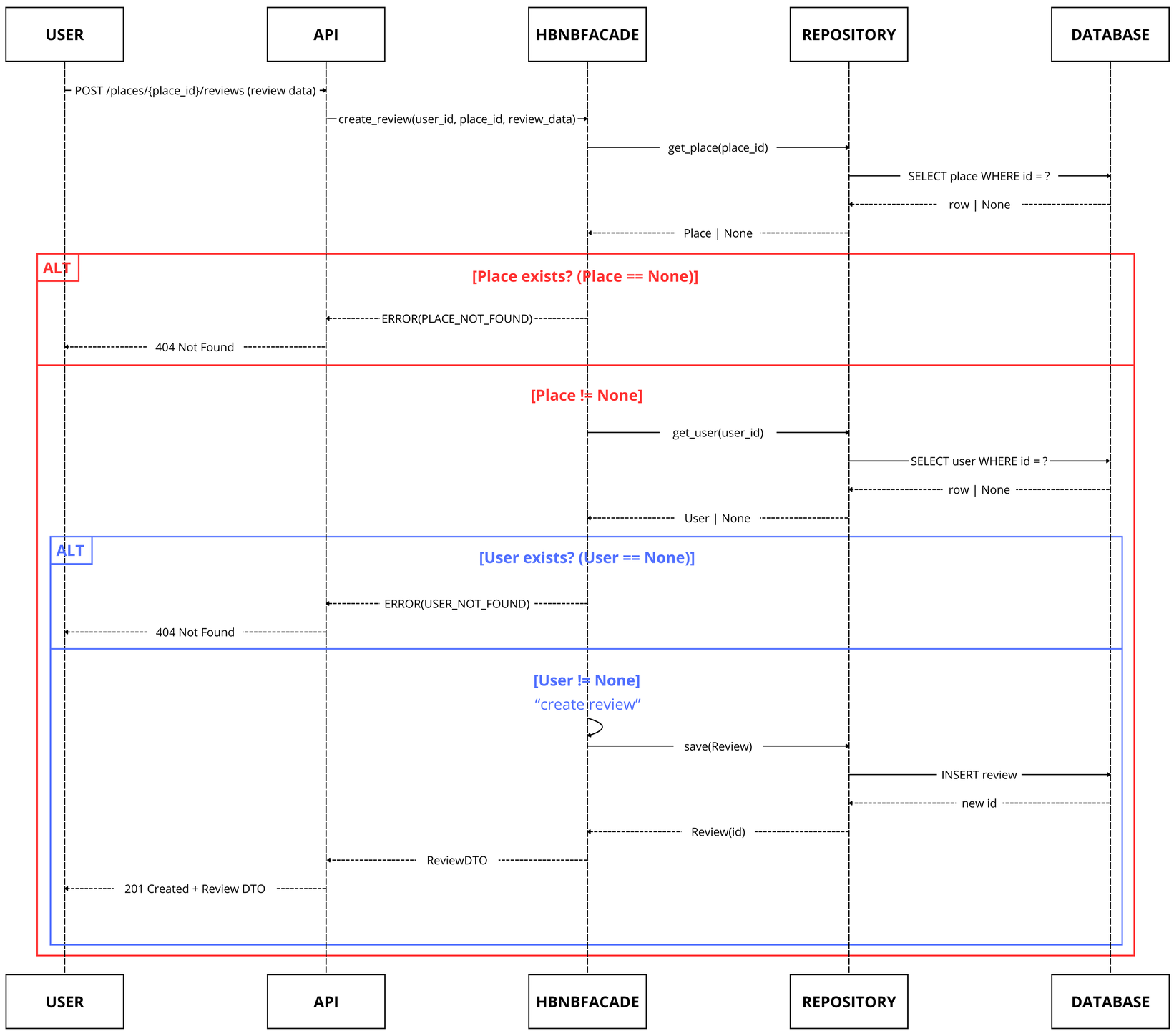
Purpose :
- Create a new place listing for an existing user (owner).

Key steps :
- User sends POST /places with place data (+ implicit/explicit user_id).
- API forwards to HBnBFacade.create_place(user_id, place_data).
- Facade validates owner exists via Repository (SELECT user WHERE id=?).
- If user not found: Facade returns Error(USER_NOT_FOUND) → API returns 404 Not Found.
- If user found: Facade creates Place entity, Repository persists it (INSERT place) and returns new id.
- Facade maps to PlaceDTO → API returns 201 Created + PlaceDTO.

Layer responsibilities :
- Presentation (API): endpoint handling, calls facade, formats HTTP response.
- Business (HBnBFacade): validates owner, orchestrates creation, maps to DTO.
- Persistence (Repository/DB): fetches user, inserts place, returns identifiers.

Outputs / errors :
- Success: 201 Created + PlaceDTO
- Failure: 404 Not Found (user not found)

SD-03 — Review Submission (POST /places{place_id}/reviews



Explanatory Notes:

Purpose :
- Submit a review linked to a specific place and a user, ensuring both exist before saving.

Key steps :
- User sends POST /places/{place_id}/reviews with review payload (+ user_id).
- API forwards to HBnBFacade.create_review(user_id, place_id, review_data).
- Facade validates place exists via Repository (SELECT place WHERE id=?).
- If place not found: Facade returns Error(PLACE_NOT_FOUND) → API returns 404 Not Found.
- If place exists, Facade validates user exists (SELECT user WHERE id=?).
- If user not found: Facade returns Error(USER_NOT_FOUND) → API returns 404 Not Found.
- If both exist, Facade creates Review entity and persists it (INSERT review) → receives new id.
- Facade maps to ReviewDTO → API returns 201 Created + ReviewDTO.

Layer responsibilities :
- Presentation (API): receives submission request and returns HTTP status + DTO.
- Business (HBnBFacade): enforces preconditions (place/user existence), orchestrates save, maps to DTO.
- Persistence (Repository/DB): performs SELECT checks and INSERT review.

Outputs / errors :
- Success: 201 Created + ReviewDTO
- Failure: 404 Not Found (place not found or user not found)

SD-04 — Fetching a List of Places (GET /places with filters)

Explanatory Notes:

Purpose :
- Return a filtered list of places.

Key steps :
- User sends GET /places with optional query filters.
- API forwards to HBnBFacade.list_places(filters).
- Facade requests Repository to query places (SELECT places WHERE filters...).
- Repository executes DB query and returns result set (rows) then Place[] entities.
- Facade maps entities to PlaceDTO[].
- API returns 200 OK + PlaceDTO[].

Layer responsibilities :
- Presentation (API): parses filters, calls facade, returns HTTP response.
- Business (HBnBFacade): orchestrates query request, maps entities to DTO list.
- Persistence (Repository/DB): executes SELECT and returns matching rows/entities.

Outputs / errors :
- Success: 200 OK + PlaceDTO[]
- Failure: typically 200 OK with empty list if no matches (unless you model invalid filters separately)